

Cómo se hizo



**UNIVERSIDAD
DE GRANADA**

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Programación Web

Profesor: Serafín Moral García

Fecha: 24 de mayo de 2026

Índice General

1	Introducción	2
2	Funciones útiles	2
2.1	PHP	2
3	Conexión a la base de datos	2
4	El paradigma en JavaScript	3
4.1	Tratamiento de asincronía	3
4.2	Las API's	4
5	Gestión de usuarios	5
5.1	Back-end	5
5.1.1	Alta de usuarios dinámica	6
6	Gestión de sesiones	7
7	El administrador	7
8	Buscador de viajes	7
9	Validación de formularios	7
10	Carrusel de viajes	7
11	Bibliografía	7

1 Introducción

A lo largo de este documento, se detallará el proceso de dinamización de la página web de *Azimuth Viajes*. Se tratará en detalle cada paso a seguir; así como los aspectos novedosos que se vayan introduciendo en el proceso de realización.

Antes de continuar debo aclarar que he decidido trabajar con el paradigma de programación dirigida a objetos para *PHP*; de manera que, siempre que se pueda se trabajará con un objeto.

2 Funciones útiles

2.1 PHP

Como cabe esperar, no todas las funciones del back-end que se necesitan pueden estar enmarcadas en una clase que contenga completa funcionalidad; por tanto, he creado un archivo *utils.php* que se encarga de albergar estas funcionalidades; sobre todo, contienen funciones para evitar repetir código o para gestionar el *html* según el tipo de usuario que usa la página web. Estas son:

- *putAvatar*: Es la encargada de escribir en la cabecera el nombre de usuario, su rol y el botón de *log out*; es decir, si es administrador aparece un ordenador y si es usuario aparece el típico avatar indeterminado.
- *putSignUpForm*: Es la encargada de escribir completamente el formulario de inicio de sesión en caso de que ésta no esté iniciada, simplemente es limpieza en el código.
- *putFormSugerencias*: Esta función tiene una funcionalidad análoga a la anterior, su utilidad es limpiar el código del archivo *sugerencias.php* y simplemente escribe el formulario de sugerencias en caso de que la sesión esté iniciada.

3 Conexión a la base de datos

En esta sección, continuamos con la gestión del back-end, es decir, tratando estrictamente con *PHP*; en este caso, hablamos de la conexión a la base de datos. En un inicio, traté de realizarlo con una clase que fuera *Conexion* y se encargará de crear y cerrar la conexión a la base de datos mediante un objeto.

Sin embargo, gracias a lo que había en el guión 3 de prácticas y a que, al realizar las demás clases, debía repetir mucho este código, decidí seguir el esquema del guión. En definitiva, he creado una clase abstracta *DataObject* que se encarga de modelar a un objeto genérico de la base de datos y del cual heredan todos.

Si nos fijamos no he hablado de las credenciales de la base de datos pero sin ellas no podemos conectarnos. Para ello, he seguido las directrices del guión 3 de prácticas que para crear el archivo *config.php* donde se "escondan" las credenciales de la base de datos y todo lo relativo a la conexión. Además, dispone de nombres genéricos para tratar de forma más simple e inequívoca las tablas de la base de datos.

4 El paradigma en JavaScript

Esta sección trata de cómo he gestionado la "comunicación" entre el *front-end* y el *back-end* y es una de las novedades o innovaciones más importantes que he implementado.

Esta se trata del paradigma **AJAJ**, evitando explicaciones es tratar la comunicación como **AJAX** pero con formato *json*. Para ello, me he basado en la gestión de la comunicación asíncrona con *API REST* donde el código *javascript* lanza peticiones *fetch* hacia una *API* para comunicarse con el *back-end*.

4.1 Tratamiento de asincronía

Tal y como he comentado en la sección anterior, estamos ante un paradigma de refresco de página sin recarga y donde *JavaScript* es el protagonista. Al igual que en cualquier comunicación, debemos mandar una pregunta y obtener una respuesta.

En el caso de una comunicación síncrona, aquel que pregunta debe esperar atentamente a que el receptor piense y de una respuesta. Sin embargo, nosotros queremos seguir trabajando mientras el receptor (*back-end*) piensa y formula su respuesta.

Para conseguir eso usaremos las siguientes palabras clave o sentencias:

- *async*: Se usa en la declaración de funciones o métodos de una clase y sirve para determinar que es un método asíncrono. Concretamente, son funciones que devuelven "promesas"; de manera que, si la función devuelve un valor, éste se envuelve en una promesa resuelta y si da error, la promesa será rechazada.

Para entenderlo más fácilmente es como si estoy hablando con una persona, le hago una pregunta y me promete que me va a responder; yo estoy trabajando en otra cosa pero tengo en mente que me tiene que dar una respuesta. Entonces, si esta respuesta es válida y correcta, la procesaré y si no lo es gestionaré el error de comunicación.

- *await*: Sólo puede usarse en entornos gestionados por *async* y se encarga de crear la promesa. Tras esta palabra suele enviarse la pregunta al *back-end* en forma de datos o suele ponerse una función para recoger los datos de respuesta.
- *fetch*: Sólo voy a explicar lo que he usado; esta sentencia es la encargada de mandar la promesa a la *API* que la procesará ¹. La sintaxis básica que yo he usado se compone de los siguientes objetos:

- *endpoint*: Dirección de la *API* a la que mandar los datos o la petición.
- *configurationObject*: Es un objeto de tipo diccionario entre llaves que se encarga de definir cómo va a ser la comunicación; algunos de sus

¹En nuestro caso la *API* será normalmente un código *php* que procesa los datos y devuelve errores o éxitos en formato *json*. Habitualmente, estarán ubicadas en la carpeta `'/php/api'`.

atributos son *method* (método *HTTP* para enviar la petición), *Content-Type* (tipo de contenido que se envía, habitualmente "application/json") y *body* (cuerpo de la petición con los datos en formato *json*).

Aunque esto puede quedar abstracto, veremos un ejemplo de uso en el alta de usuarios. Por último, y a modo de resumen, podemos ubicar que la metodología para trabajar con *fetch* es la siguiente:

1. Definir el método *async*.
2. Creamos la promesa donde nos comunicamos con la *API* a través de *fetch*.
3. Comprobamos si hay respuesta de la *API* y la conexión ha ido como debería². En caso contrario gestionamos el error³.
4. "Esperamos" a recibir la respuesta al mensaje completo en formato *json*.
5. Procesamos los datos ya habiendo finalizado la consulta asíncrona.

En definitiva, es un proceso más o menos sencillo igual que otro. En clase vimos cómo hacerlo con *xmlHttpRequest*; sin embargo, buscando información encontré que este método es más novedoso y se está utilizando actualmente en el desarrollo web. Desconozco que el paradigma visto en clase se siga o no utilizando pero, es cierto, que me pareció más complicado. Por eso, elegí este paradigma; además, es una nueva innovación.

4.2 Las API's

Habitualmente, estas *API's* ya están creadas y con plena funcionalidad para ser usadas por los programadores de una forma específica. Sin embargo, yo debo crearlas. Tal y como comenté en un pie de página en la sección 4.1 están ubicadas en la carpeta 'php/api/' pues no son más que ficheros que se encargan de procesar una entrada en formato *json* y devolver una salida, de nuevo, en formato *json*.

En estas *API's* se debe crear un mensaje *HTTP* al completo con la información a enviar. Las partes que se tratan son:

- *Header*: con la sentencia *Header()* podemos crear y formatear la cabecera del mensaje *HTTP* que vamos a enviar como respuesta.
- *Obtención de datos*: aquí procesamos los datos que debemos recibir; en nuestro caso, usamos la siguiente sentencia:

Donde, debido a que *PHP* no puede leer los datos del *body* accediendo a la variable *\$_POST*, accedemos al flujo de entrada en crudo ("php://input"). Además, 'json_decode' se encarga de decodificar el formato y pasarlo a un *array* asociativo de *PHP*.

²Lo que realmente estamos procesando es la respuesta de la *API* al recibir la cabecera de nuestro mensaje.

³Habitualmente lanzando un error.

- *Procesamiento de datos*: en esta parte, procesamos los datos, cada *API* lo hará a su manera.
- *Enviamos respuesta*: fase en la cual debemos responder a la petición. Habitualmente, se utiliza la siguiente sentencia:

La sentencia *echo* es la forma nativa de escribir datos en el cuerpo de una respuesta *HTTP*; por tanto, estamos enviando la respuesta tras cerrar la ejecución del *script* y codificando en modo *json* el *array* asociativo con las respuestas.

Por tanto, ya hemos visto todo lo necesario para trabajar con el *front-end* y *back-end* de forma asíncrona con *fetch*.

5 Gestión de usuarios

Entramos ya en aspectos importantes, una aplicación web no es nada sin usuarios; por tanto, debemos crear una tabla para ellos en la base de datos. Dicha tabla ('Users') se encarga de modelar a un usuario que dispondrá de los siguientes atributos:

- Nombre
- NickName (PK)
- Email (Unique)
- Password: Este atributo siempre debe aparecer cifrado para que no sea visible para ninguna persona con intenciones maliciosas.
- Admin: Este atributo refleja si el usuario es administrador o no, por defecto, tomará el valor 0 pues el único administrador ya está creado en la base de datos y es el usuario cuyo nickname es 'el_dramas1s'.

Con esta representación podremos gestionar a los usuarios de forma más o menos cómoda.

5.1 Back-end

Con respecto al cómo se ha modelado para el trato en el servidor (*PHP*), he creado una nueva clase en el archivo *user.php* que hereda de *DataObject* y que modela exactamente las características de los usuarios.

Esta clase, que al principio modelé como una clase instanciable y otra clase distinta que lo manejaba, se encarga ahora de hacer ambas cosas usando métodos de clase y métodos de objeto.

Referente a los métodos de objetos, son los encargados de gestionar las operaciones de la base de datos, así como de realizar algunas verificaciones de los objetos en

cuestión. Estos métodos crean usuarios en la base de datos, los eliminan, los buscan y verifican que la contraseña y el *nickName* pasados como argumentos estén asociados.

Para el caso de los métodos de objeto, tenemos un método *getter* para obtener los datos, un método de validación de consistencia de un usuario y un método para comprobar si es admin.

5.1.1 Alta de usuarios dinámica

Comienza lo interesante, debemos ahora gestionar la alta de nuevos usuarios en la página web; por tanto, deberemos tener en cuenta que debemos comprobar el formulario de alta de usuario. Esta tarea no solo es del front-end sino que también del back-end pues nos dará mayor protección frente a fallos del usuario.

Por tanto, he creado un archivo *forms.php* que se encarga de manejar los formularios. Este archivo sigue la misma lógica de árbol de herencia que los *DataObjects* pues disponemos de un manejador que se encarga de modelar el comportamiento básico como una clase abstracta que aparece en el *listing ??*.

Donde aparecen las cuatro funciones básicas de un formulario:

- **Validar entradas:** La función *validate* se encarga de validar las entradas del objeto, cada formulario tendrá sus validaciones dependiendo del objeto que se esté tratando.
- **Procesar entradas:** La función *process* se encarga de procesar el formulario dándole la funcionalidad que merece. Dependiendo del formulario que se trate realizará unas operaciones u otras.
- **Gestionar posibles errores:** Esta funcionalidad la realizan dos funciones, *addError* y *getErrors*, junto con el dato miembro de la clase que es el array de errores.

Por tanto, cada vez que queramos gestionar un formulario, lo haremos a través de la función *handle* que engloba las dos primeras. Y si devuelve *false* procesaremos los errores con *getErrors*.

Nos centramos ya en el formulario de alta de usuarios, cuyo manejador es *Form-SignUp* que se encarga de obtener los datos, validarlos y, si cumple las características básicas para poder crear un usuario, se crea en la base de datos. La única característica que queda fuera de las básicas es una condición de política; esta es: "un usuario no puede ser menor de edad"⁴ ⁵.

El código de este manejador aparece en el *listing ??*.

⁴El motivo de esto es evitar pagos de usuarios menores de edad, en caso de falsificarlo es problema del usuario.

⁵Si nos fijamos, no había ningún atributo de mayoría de edad en la base de datos para el usuario; el motivo es que si está en la base de datos es porque se mayor de edad.

Fijándonos ya en el archivo *altausuarios.php*, puedo determinar que no hace falta usar el archivo *validacion.html* para mostrar que el usuario se ha creado correctamente, pues con programación en el back-end es posible realizarlo. El código de esta mejora se encuentra, como esqueleto ⁶ en el *listing ??*.

Como podemos ver, hay dos partes diferenciadas, comentemos cada una de ellas:

- **Primera.** Esta sección se encarga de gestionar si se entra o no en el formulario y en la sección de errores. El array *\$_SERVER* es un array que contiene información sobre el servidor, concretamente estamos mirando si el método de acceso a la página es *POST*. En caso afirmativo, suponemos que el formulario se ha mandado y lo procesamos.

Ahora bien, si hubiera habido algún fallo o no se ha mandado el formulario, debemos mostrarlo otra vez y mostrar también los errores.

- **Segunda.** Esta sección muestra que todo ha salido bien y que el usuario se ha creado correctamente, es decir, el código de *validacion.html*.

Lo último por comentar del control de alta de usuarios en el back-end es cómo he hecho para guardar los datos que el usuario introdujo en caso de que fallara el alta de usuario. Para ello, he usado el array *\$_POST* que vimos que contiene los datos que se enviaron en el formulario por última vez; de forma que, si no está vacío lo incluyo en el campo *value* ⁷. El código para el caso del *nickName* se encuentra en el *listing ??*.

6 Gestión de sesiones

7 El administrador

8 Buscador de viajes

9 Validación de formularios

10 Carrusel de viajes

11 Bibliografía

⁶Me refiero, sin el *html* interno.

⁷En el caso de las contraseñas no por seguridad.